
CIPHER: A Benchmark for Inverse Transformation Program Recovery in Multimodal Agents

Maxim Kurkin
kurkin@2a2i.org

Abstract

We introduce **CIPHER**, a procedurally generated benchmark for *inverse transformation program recovery* in tool-using multimodal systems. In CIPHER, a model receives a corrupted video observation together with natural-language instructions describing the transformations required to restore it. The system must (i) infer the latent transformation program, (ii) realize the inverse tool chain via `ffmpeg`, and (iii) localize and extract a short target string that is readable only after successful recovery. This yields a stage-decomposed evaluation of **program inference** (S1), **tool-chain execution** (S2), and **terminal-state extraction** (S3), rather than a single monolithic score.

CIPHER spans five calibrated difficulty levels that vary key visibility, transformation-chain length, instruction ambiguity, and distractor density, with deterministic seed-based replay and procedural regeneration for contamination resistance and reproducible regression testing. We also instantiate a DAVIS-2017 video-temporal split-key domain that measures multi-frame aggregation and operation inversion with dense process partial credit. We evaluate OCR baselines, heuristic pipelines, and frontier multimodal models. Current systems are strong in S3-only extraction when given the correct terminal frame, but remain fragile in end-to-end recovery where temporal localization, transformation sequencing, and executable tool use must all be correct simultaneously. CIPHER is therefore best viewed as a diagnostic benchmark for inverse procedural reasoning in multimodal agents, not merely as another passive video-understanding leaderboard.

1 Introduction

Recent multimodal models are increasingly strong at passive video understanding: they can answer questions about actions, summarize clips, and describe temporal events with surprising fluency. But many agentic failures do not occur at the level of recognizing what is in a video; they occur at the transition from *understanding* to *executable recovery*. A system may understand that a video has been mirrored, reversed, or sped up, yet still fail to realize the correct inverse tool chain, search the right temporal window, or extract the target signal from the restored result.

CIPHER targets exactly this gap. It instantiates a narrow but revealing setting: given a corrupted video observation and natural-language instructions, a system must infer, invert, and execute a latent transformation program, then localize and extract the target signal. The surface form resembles a video-editing challenge, but the underlying capability is *inverse multimodal procedural reasoning* under tool use.

Despite the proliferation of video-capable VLMs [OpenAI, 2023, Google, 2023, Wang et al., 2024], no benchmark systematically tests this compound capability. While current video understanding benchmarks such as Video-MME [Fu et al., 2024] and LongVideoBench [Wu et al., 2024] assess passive temporal reasoning and question answering, they do not require active, sequential manipulation of the visual stream based on language. Conversely, video generation and editing benchmarks

like VBench [Huang et al., 2023] evaluate perceptual quality rather than the precise execution of rigid transformational rules. Neither paradigm captures the tight coupling of instruction-following, procedural tool use, and visual extraction required here.

Two design choices make CIPHER deliberately interpretable. First, the transformation vocabulary is intentionally minimal and reversible: `reverse`, `hflip`, `vflip`, `rotate90`, `rotate270`, and `speed2x`. This keeps failures attributable to reasoning and execution rather than to an uncontrolled action space. Second, evaluation is stage-decomposed into program inference (S1), tool-chain execution (S2), and terminal-state extraction (S3), making it possible to ask *where* a system fails rather than merely whether the final string is correct.

Contributions.

1. **Benchmark for inverse transformation program recovery.** We formulate a multimodal task in which the model must recover a latent transformation program from corrupted video observations and natural-language instructions, then execute the inverse chain and extract the target signal.
2. **Stage-decomposed evaluation.** We decompose the task into program inference (S1), tool-chain execution (S2), and terminal-state extraction (S3), allowing pipeline-stage attribution instead of a single opaque end-to-end score.
3. **Procedural benchmark infrastructure.** CIPHER provides deterministic seed-based generation, contamination resistance, replayable regression cases, and a calibrated difficulty ladder over visibility, chain length, instruction ambiguity, and distractor density. We additionally introduce a video-temporal split-key domain over DAVIS-2017 clips for fast iteration on temporal aggregation and operation inversion, together with a reproducible skill-driven evolution pipeline for mining valid hard samples.
4. **Evidence of an execution gap.** Across OCR baselines, heuristic pipelines, and frontier multimodal models, strong S3-only extraction does not translate into reliable recovery. The current video-temporal slice further shows that many failed answers contain meaningful partial progress, exposing a gap between perception, temporal evidence selection, and executable policy competence.

2 Task Definition

Setting. Let $\mathcal{V}$ be the space of video clips and $\mathcal{L}$ the space of natural-language strings. CIPHER evaluates a function $f : \mathcal{V} \times \mathcal{L} \rightarrow \Sigma^7$ that maps a corrupted video v' and an instruction $\mathcal{I}$ to a predicted target string $\hat{k}$. The original clip v contains a latent key $k^* \in \Sigma^7$ embedded during a temporal window $[t_s, t_e]$. A sequence of transformation operations $\mathbf{o} = (o_1, \dots, o_n)$ is applied to v to yield $v' = o_n \circ \dots \circ o_1(v)$. The agent’s objective is to recover the inverse program $\mathbf{o}^{-1}$ from $\mathcal{I}$ and v' , execute it, and extract k^* from the recovered terminal state.

The key k^* is drawn from unambiguous alphanumerics ($\Sigma = \text{A-Z} \cup \text{2-9} \setminus \{\text{O,0,I,1}\}$). Instructions are generated from $\mathbf{o}$ via paraphrase templates.

Three-stage pipeline. This task is intentionally composite. An end-to-end failure ($\hat{k} \neq k^*$) may arise because the model inferred the wrong program, realized the wrong tool chain, or failed to localize and read the key after successful recovery. By enforcing a three-stage pipeline—Instruction Parsing (S1), Execution (S2), and Key Extraction (S3)—CIPHER isolates these failure modes. In policy terms, S1 corresponds to *program inference*, S2 to *tool-chain realization*, and S3 to *terminal-state extraction*. This decomposition prevents monolithic metrics from masking which part of the reasoning policy is actually failing.

S1 Parse: $\mathcal{I} \rightarrow \hat{\mathbf{o}}$. Score: sequence-aware op F1.

S2 Execute: run `ffmpeg`($\hat{\mathbf{o}}$). Score: binary validity.

S3 Extract: read $\hat{k}$ from edited video. Score: CharF1 + EM.

Operations. The operation vocabulary is intentionally minimal: six composable `ffmpeg` transforms, `reverse`, `hflip`, `vflip`, `rotate90`, `rotate270`, and `speed2x`. These are reversible, visually interpretable, and cover both spatial and temporal recovery. The restricted vocabulary is a design choice: it keeps the benchmark focused on inverse program recovery rather than on open ended action-space exploration.

Video-temporal diagnostic domain. In addition to the original inverse-`ffmpeg` pipeline, we instantiate a controlled video-temporal split-key domain for faster iteration on temporal aggregation. A key is split into two fragments that appear on two consecutive frames of a short clip sampled from the public DAVIS-2017 video corpus [Pont-Tuset et al., 2017]. The solver must identify the target frames, read both fragments, and invert a small symbolic operation before answering. The operation set is `none`, `swap_fragments`, `reverse_chars`, and `swap_reverse_chars`; the last two require character-level transformation rather than simple OCR.

This domain deliberately separates *temporal evidence acquisition* from generic video editing: the latent program, target frame indices, fragment bounding boxes, displayed fragments, normalized fragments, and final answer are all known from generation. This enables dense process scoring. Besides strict EM, we assign partial credit for operation recognition, target-frame localization, displayed-fragment similarity, normalized-answer similarity, transform-derived answer similarity, final-answer similarity, and successful tool use. The resulting traces are used both for model diagnosis and for evolutionary seed selection.

3 The CIPHER Benchmark

CIPHER is designed as a benchmark for inverse procedural reasoning rather than as a static collection of adversarial examples. The generator exposes independent stress axes for policy recovery, while deterministic seeds make every failure replayable.

3.1 Difficulty levels

Table 1: CIPHER difficulty levels (50 samples each, $N=250$ total).

Level	Key vis.	Ops	NL amb.	Distr.	Primary challenge
L1	1 (large)	1	1	0	Single-op sanity check
L2	1 (large)	2	1	0	Two-op program recovery
L3	2 (medium)	2	2	1	Paraphrase + distractor pressure
L4	2 (medium)	3	2	2	Longer chain + distractors
L5	3 (small)	4	3	3	Compositional full difficulty

Key visibility varies font size (13%→7% of frame height), alpha (255→200), and position randomization (centered→random). The key is always rendered in bright yellow with a black drop shadow; distractors use non-yellow colors and are always smaller and dimmer.

Capability axes. The benchmark factors difficulty into independent axes that correspond to distinct stresses on the recovery policy: key visibility controls terminal extraction difficulty; operation count controls latent-program length; instruction ambiguity controls inference difficulty; and distractor density controls false terminal-state attraction. This design supports attribution of performance drops to specific parts of the policy rather than to a single undifferentiated “harder” setting.

NL ambiguity levels. (1) exact canonical phrasing; (2) paraphrase from a pool of templates per operation; (3) compositional “undo” framing with embedded temporal cues. All phrases are intentionally designed to remain scoreable for S1 baselines.

Operation vocabulary. The operational vocabulary consists of six composable transformations: `reverse`, `hflip`, `vflip`, `rotate90`, `rotate270`, and `speed2x`. These operations are visually distinct, non-destructive, and composable via standard `ffmpeg` filters. The vocabulary is intentionally small to preserve interpretability.

3.2 Procedural generation and dataset integrity

Samples are fully deterministic from (*seed*, *level*). Background clips are drawn from a Kinetics-400 pool, and the manifest stores the generated sample metadata explicitly.

A persistent challenge in evaluating large language and vision models is the risk of data contamination, where static benchmark data leaks into the pre-training corpus. CIPHER mitigates this through programmatic dataset synthesis. The benchmark is generated from (*seed*, *level*), which deterministically selects the background clip, random key, operator chain, and instruction template. Because samples are constructed on demand using disjoint background pools, researchers can regenerate the benchmark with novel seeds, guaranteeing an uncontaminated evaluation environment and a renewable supply of test instances. The same mechanism also makes failures replayable: a single seed is sufficient to reconstruct the exact corrupted observation, latent program, and target timing window for regression-driven debugging.

3.3 Difficulty calibration via GEPA

A benchmark that either saturates (all models at 100% EM) or collapses (all at 0% EM) provides little discriminative signal. Rather than targeting a single operating point, we calibrate a ladder of five extraction conditions, L1–L5, such that a reference frontier VLM spans the full accuracy range from near-ceiling to near-floor. This design supports informative scaling curves and helps preserve discriminative power as models improve.

To find the five conditions we adopt a GEPA-style [Khattab et al., 2025] reflective evolutionary loop over the image-mode generator. This search calibrates *benchmark difficulty*, not model weights. The generator is an S-expression DSL with tunable numerical parameters including key alpha, font size, distractor count and color, Gaussian noise, and blur radius. Each *program* is a fixed instantiation; *fitness* is the squared error between observed EM and a per-level target profile.

We intentionally run this search first in image mode rather than in the full video pipeline. Direct video-level calibration mixes extraction difficulty with pipeline noise from parsing, tool execution, and temporal localization. Image mode yields a cleaner fitness signal for terminal-state readability, after which the resulting ladder can be transferred into the full procedural benchmark.

3.4 Skill-driven evolution pipeline

The benchmark is generated through a reproducible skill-driven evolution pipeline rather than by manual adversarial editing. Each run is anchored by a git commit, a YAML configuration, a registered set of mutable slots, and an EvoMCP trace bundle. The pipeline exposes two mutation surfaces. GEPA mutates only the text genome—prompts, rubrics, and response protocols—while EvoX mutates only the program genome—generator parameters, temporal-operation choices, backend lanes, and scoring weights. This separation is important: it keeps changes to task semantics distinct from changes to the evaluator or agent framing.

Operationally, a run begins by verifying the registered slots, then executing the corresponding skill command, e.g. `/evolve evox configs/evox_video_temporal_davis_validity_tuned.yaml`. The local equivalent is the project wrapper `scripts/evomcp_cipher.py`, which routes EvoX through the same CIPHER evaluator used by the MCP server. Each candidate writes `candidate.json`, `inputs.json`, `result.json`, event logs, and generated media under an artifact directory. These trace bundles are the unit of replay, reporting, and paper-table provenance.

For the video-temporal split, we use a two-stage schedule. The first stage is validity-first calibration on DAVIS-2017 480p clips [Pont-Tuset et al., 2017], using moderate font size, opacity, jitter, frame count, and noise. Its objective targets readable samples, low missing-answer rate, and nonzero exact match, so that both EM and process partial credit provide usable signal. The second stage performs hard mining from the resulting seedbank. It does not keep only the scalar best candidate: seeds are selected from all candidate traces using validity, non-missing rate, low EM after validity is established, process partial credit, frame-localization credit, operation-component credit, transform/final-answer similarity, and diversity over temporal operations. This prevents the benchmark from collapsing into invalid noise while still driving the pool toward frontier-model failures.

The current released pipeline is encoded in three committed configs: a DAVIS validity-tuned EvoX pass, a DAVIS weak-seedbank/process hard-mining pass, and a Hermes subscription campaign for Codex, Claude, and Gemini lanes. The repository also contains a collaborator runbook, docs/EVOLUTION_PIPELINED.md, which specifies the exact filenames, slot checks, remote skill mapping, seedbank policy, artifact contract, and report regeneration commands.

Table 2: Calibrated extraction levels, measured on Claude Sonnet 4.6 (reference model during calibration, $n=8-16$ per level).

Level	Config (α , font%, dist, σ , blur)	target EM	obs. EM	char-acc
L1	255, 12%, 0, 0, 0	95%	62%	94%
L2	240, 10.5%, 0, 0, 0	70%	69%	94%
L3	225, 9%, 0, 0, 0	40%	44%	92%
L4	215, 8%, 1c, 0, 0	15%	25%	82%
L5	205, 7.5%, 1 (1y), 0.5, 0	3%	0%	55%

3.5 Metrics

EM. $\frac{1}{N} \sum_i \mathbf{1}[\hat{k}_i = k_i^*]$.

CharF1. Character-level precision/recall F1 for near-miss credit.

PipeF1.

$$\text{PipeF1} = 0.25 \cdot \overline{S1} + 0.25 \cdot \overline{S2} + 0.50 \cdot \overline{\text{CharF1}} \quad (1)$$

S3 receives double weight as the terminal objective; S1 and S2 are diagnostic leading indicators for policy failure attribution.

4 Experiments

4.1 Experimental setup

Inference modes. We evaluate models in two settings: (1) **S3-only**: models receive a ground-truth corrected frame containing the key and must read it. This tests pure terminal-state extraction and serves as an upper-bound control slice. (2) **Full pipeline**: models receive the scrambled .mp4 video and natural-language instructions with Bash access, and must independently parse instructions (S1), execute ffmpeg corrections (S2), locate the key window, and extract the key (S3).

The distinction is central: strong performance in S3-only does not imply strong inverse program recovery. It only shows that once the correct terminal state is provided, the model can decode the target string. Full-pipeline performance measures whether the system can actually reach that terminal state.

Agentic evaluation. For full-pipeline evaluation, VLM agents are given Bash access and must decide their own video processing strategy (ffmpeg flags, frame sampling, temporal search). No ground-truth metadata (operations, key, timing) is provided.

Reported metrics. We report strict exact match (EM), character-level F1, stage-specific S1 parse F1, S2 execution success rate, and Pipeline-F1. In the current draft, the strongest supported evidence comes from S3-only and calibrated extraction experiments; full-pipeline reporting remains incomplete and is therefore presented as an explicit gap rather than as a filled leaderboard.

For the video-temporal diagnostic domain, we additionally report *process partial credit*. This score decomposes a failed answer into operation credit, frame-localization credit, displayed-fragment similarity, normalized-answer similarity, transform-derived-answer similarity, final-answer similarity, and tool-use credit. It is intentionally not a replacement for EM; it is a dense diagnostic signal used to distinguish uninformative failures from meaningful near misses.

4.2 Models

We benchmark OCR baselines, heuristic pipelines, and frontier multimodal systems:

OCR-only Pure extraction baseline. Uses easyocr on sampled key frames; no language understanding, program inference, or tool use.

Yellow-OCR Color-filtered OCR baseline. Isolates yellow pixels via HSV masking before running easyocr. Tests whether color-aware preprocessing can substitute for reasoning or executable recovery.

MCP Skill Heuristic agentic baseline (`solve_captcha.py`): regex-based instruction parsing (S1), ffmpeg execution (S2), and a multi-method extraction chain (S3).

Claude Haiku / Sonnet / Opus API-first frontier multimodal systems used to probe the gap between prepared extraction and end-to-end recovery.

GPT-5.4 Tool-using frontier model evaluated through the same full-pipeline protocol.

4.3 Current results

Table 3: S3 key extraction from ground-truth corrected frames ($n=5$ per level). Best per column in bold.

Model	L1	L2	L3	L4	Total
OCR-only	20%	25%	30%	25%	25%
Yellow-OCR	33%	—	—	—	—
MCP Skill	20%	25%	30%	25%	25%
Haiku	100%	60%	40%	100%	75%
Sonnet	100%	100%	100%	100%	100%
Opus	100%	100%	100%	100%	100%

Table 4: Full pipeline results (video $\rightarrow$ key). This table is intentionally left incomplete in the current draft; filling it is a required next experiment because the benchmark’s main claim concerns the gap between S3-only extraction and end-to-end recovery.

Model	L1 EM	L2 EM	L3 EM	L4 EM	EM (all)	PipeF1
MCP Skill	—	—	—	—	—	—
Haiku	—	—	—	—	—	—
Sonnet	—	—	—	—	—	—
Opus	—	—	—	—	—	—
GPT-5.4	—	—	—	—	—	—

Overview. Table 3 reports S3 extraction accuracy in isolation: each model receives a corrected frame containing the yellow key and must read it. Frontier VLMs (Opus, Sonnet) achieve perfect accuracy, while Haiku drops on smaller, randomly positioned keys. OCR baselines perform poorly due to background text interference and clutter.

Crucially, S3 in isolation is *not* the binding bottleneck for frontier models. The benchmark’s real difficulty lies in the full recovery policy: agents must infer the inverse transformation program, execute it correctly, and identify the correct temporal window containing the key. The missing full pipeline table is therefore not a cosmetic omission; it is the central remaining experiment required to validate the benchmark’s main claim.

4.4 Video-temporal diagnostic results

We also ran a controlled DAVIS-2017 video-temporal split in which the model must aggregate two consecutive frames and invert a symbolic fragment transformation. This split is not yet the full inverse-ffmpeg benchmark; it is the current fast iteration domain for studying temporal aggregation, solvability, and dense process metrics. The current pool contains 78 generated tasks with process

metrics and 233 model responses across three CLI backends. All modern samples in this slice use dynamic DAVIS frame sequences and pass the current “likely-readable” solvability gate.

Table 5: Current DAVIS video-temporal diagnostic slice. EM is strict exact match. Missing counts empty/NONE final answers. Partial is dense process partial credit. Values are means over response rows; $\pm$ is population standard deviation.

Backend	n	EM $\uparrow$	Missing $\downarrow$	Partial $\uparrow$
Claude CLI Haiku	78	14.1 $\pm$ 34.8	1.3 $\pm$ 11.2	43.1 $\pm$ 28.6
Codex CLI GPT-5.3 Spark	77	0.0 $\pm$ 0.0	100.0 $\pm$ 0.0	6.5 $\pm$ 5.6
Gemini CLI Flash Lite	78	0.0 $\pm$ 0.0	98.7 $\pm$ 11.2	5.0 $\pm$ 5.7

At the task level, 11 of 78 tasks are *strict-good*: at least one model achieves EM. A larger set of 45 tasks is *semantic-good*, meaning either EM or maximum process partial credit at least 0.25. Finally, 77 tasks have at least one non-missing response with partial credit at least 0.15, making them usable for debugging even when no backend solves them exactly. The gap between strict-good and semantic-good is the most important observation: the split contains many failures that are not random, but partially correct trajectories.

The strongest current backend is Claude CLI Haiku, which often identifies the operation and produces structured intermediate evidence but still loses characters or over-reads distractors on hard samples. Codex Spark and Gemini Flash Lite mostly fail by returning no final answer in this setup, although their traces occasionally receive small operation/tool credit. This suggests that backend/tool framing and response protocol are part of the benchmark surface, not merely implementation details.

4.5 Required experiments for the camera-ready version

The current draft still lacks the analyses that best express CIPHER’s identity as a diagnostic benchmark:

- scaling curves over chain length, instruction ambiguity, distractor density, and temporal sparsity;
- paired S3-only vs full-pipeline comparisons with an explicit execution-gap metric;
- stage-attribution plots for S1, S2, and S3 across model families; and
- a curated failure-seed suite with traces.

The video-temporal slice supplies the first version of this failure-seed suite: 12 seedbank genomes were selected by validity, non-missing rate, process partial credit, frame-localization credit, transform similarity, final-answer similarity, and diversity. The selected genomes cover all four fragment operations and range from easy validity-tuned settings (4–6 frames, low noise, large bright text) to hard-mined settings (7–8 frames, high jitter, distractors, and `swap_reverse_chars`).

5 Analysis

CIPHER is most valuable when failures are treated as structured signals rather than as a flat collection of incorrect strings. The analysis should therefore be read as a *failure geometry*: which stage breaks first, how errors change with difficulty, and which seeds should be preserved as regression cases. Some subsections below remain partially prospective pending full-pipeline results, but the intended analytical frame is already clear.

5.1 Stage bottleneck analysis

We decompose failures by stage to identify where the recovery policy breaks down.

S1 (program inference). On simple levels, S1 often saturates first: regex matching achieves high parse F1 on canonical instructions. The remaining errors are not random; they concentrate in paraphrastic or compositional instructions where the system must recover the latent operator sequence rather than match surface forms.

S2 (tool-chain execution). Execution failures are rare on short chains but increase with operation count, primarily because the model must realize a valid `ffmpeg` chain rather than merely name the right operations.

S3 (terminal-state extraction). S3 is *not* the binding bottleneck for frontier VLMs once the correct frame is provided. The true difficulty lies in the coupling between temporal localization and execution: many systems that can read the key still fail to arrive at the correct frame.

Near-miss errors are informative. Near-miss predictions should not be discarded as noise. They reveal whether a model reached the correct terminal state but misread one character, localized the wrong frame, or recovered the wrong but visually plausible distractor. In a diagnostic benchmark, these distinctions matter more than a single binary EM label.

5.2 Character-level confusion in the near-miss regime

Under calibrated extraction settings, models produce consistent substitutions among visually similar glyphs rather than random strings. These errors are useful because they identify a readable-but-fragile regime in which the task is still discriminative.

5.3 Video-temporal failure taxonomy

The DAVIS video-temporal slice makes the failure geometry concrete. Among the 78 modern video-temporal tasks, 67 currently have no exact match from any evaluated backend, but only one task is an all-missing failure across all models. This is the desired hard-but-diagnostic regime: most failures still contain usable intermediate evidence.

The dominant failure families are:

- **near misses:** 21 tasks have no EM but reach maximum process partial credit at least 0.5, indicating that a model recovered part of the operation/frame/fragment structure but failed the final string;
- **frame-localization weakness:** 24 tasks have maximum frame-localization credit below 0.5, showing that models often read from non-target frames or from the entire clip rather than the two target frames;
- **over-reading:** 12 tasks contain predictions longer than the gold key by more than two characters, usually because the model absorbs distractors or neighboring yellow text into the answer; and
- **hard swap-and-reverse cases:** the highest-difficulty frontier is dominated by `swap_reverse_chars` seeds with 7–8 frames, high jitter, distractors, and non-central placement.

This taxonomy directly motivates the dense metric. A flat EM score would mark all of these examples as identical failures. Process partial credit separates “no evidence”, “found the operation”, “localized the wrong frame”, and “read the right evidence but transformed it incorrectly”, which are different failure modes for an agentic multimodal system.

5.4 Canonical failure suite

For benchmark-driven development, we recommend preserving a curated suite of 20–50 canonical failure seeds with associated traces. Each entry should store: seed, level, background regime, gold operator chain, predicted operator chain, execution trace, temporal localization decision, predicted key, and error family. This turns CIPHER from a static benchmark into a replayable regression environment for agentic systems.

The current video-temporal seedbank is the first implementation of this idea. It stores not only the winning candidates, but also diverse non-winning candidates selected by a utilization–diversity criterion. This matters because the most useful benchmark samples are not always the hardest candidates by EM: some are slightly easier but much more interpretable, and some expose specific operation or temporal-localization weaknesses that would be lost if we kept only the scalar best-of-run candidate.

6 Conclusion

We introduced CIPHER, a procedurally generated benchmark for inverse transformation program recovery in tool-using multimodal systems. CIPHER asks whether a model can infer, invert, and execute a latent transformation program from corrupted video observations and natural-language instructions, then localize and extract the target signal. Its three-stage decomposition and Pipeline-F1 metric enable fine-grained diagnosis of where the recovery policy breaks down.

Our current results already show a strong asymmetry: frontier VLMs solve S3 key extraction near-perfectly in isolation, while OCR baselines struggle. The more important open question, and the central claim CIPHER is designed to measure, is the gap between this prepared extraction competence and reliable end-to-end policy execution under temporal search and tool use. The benchmark is therefore most useful not as a flat leaderboard, but as a diagnostic environment for stage-attributed failure analysis and reproducible regression testing.

The new DAVIS video-temporal slice strengthens this claim. Even before the full inverse-ffmpeg table is complete, the split-key domain shows that strict EM alone hides useful structure: many samples are not solved exactly, yet model traces contain partially correct operation, frame-localization, or fragment-normalization evidence. This supports CIPHER’s core design principle: the benchmark should preserve process traces and dense diagnostics rather than compress all behavior into a single final-string score.

Limitations. CIPHER focuses on a small deterministic ffmpeg vocabulary; generative edits and audio-only keys are out of scope. Human baselines are not included in this version.

Future work. The most important next steps are empirical rather than nominal: complete the paired S3-only vs full-pipeline table, add scaling curves over chain length, instruction ambiguity, distractor density, and temporal sparsity, and curate a canonical failure-seed suite with traces. Beyond that, temporal edit expansion, multi-frame extraction, and stronger policy-search baselines are natural extensions.

For the video-temporal domain specifically, the immediate target is a frozen pool of roughly one hundred audited valid samples, balanced across none, swap_fragments, reverse_chars, and swap_reverse_chars. This pool should be evaluated with Haiku/Sonnet/Opus, Codex, and Gemini backends under paired RPAD-style debiasing so that hardness estimates are not dominated by one model family’s response format or tool-use behavior.

Reproducibility. All code and generation scripts are released at <https://github.com/Fr0do/cipher>. The benchmark generator is deterministic by seed and level, enabling replay of individual failure cases. The release also includes the evolution runbook used for the current DAVIS video-temporal slice: slot verification, EvoMCP skill commands, validity-first calibration, seedbank selection from all candidates, hard-mining configs, trace-bundle layout, and report regeneration are documented in docs/EVOLUTION_PIPELINES.md.

Acknowledgements

References

- Chaoyou Fu et al. Video-mme: The first-ever comprehensive evaluation benchmark of multi-modal llms in video analysis. In *CVPR*, 2024.
- Google. Gemini: A family of highly capable multimodal models. *arXiv:2312.11805*, 2023.
- Ziqi Huang et al. Vbench: Comprehensive benchmark suite for video generative models. *arXiv:2311.17982*, 2023.
- Omar Khattab et al. Gepa: Reflective prompt evolution for program optimization. *arXiv:2507.19457*, 2025.
- OpenAI. Gpt-4 technical report. *arXiv:2303.08774*, 2023.

Jordi Pont-Tuset, Federico Perazzi, Sergi Caelles, Pablo Arbeláez, Alexander Sorkine-Hornung, and Luc Van Gool. The 2017 davis challenge on video object segmentation. *arXiv:1704.00675*, 2017.

Peng Wang et al. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv:2409.12191*, 2024.

Haoning Wu et al. Longvideobench: A benchmark for long-context interleaved video-language understanding. *arXiv:2407.15754*, 2024.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: Claims about label-free reward and benchmark performance are supported by the Section 5 experiments on MATH, HumanEval/MBPP, and GORE.

Guidelines:

- The answer [\[NA\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[NA\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: The limitations section notes the $O(d^2)$ Hamiltonian construction cost, noisy reward for very short chains, and asymptotic theoretical guarantees.

Guidelines:

- The answer [\[NA\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.

- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[Yes\]](#)

Justification: Section 4 proves the connection to energy-based models and optimal transport and states all assumptions explicitly.

Guidelines:

- The answer [\[NA\]](#) means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [\[Yes\]](#)

Justification: Sections 3–4 specify the procedural generator, deterministic seeds, public DAVIS-2017 source clips, evaluation metrics, and the skill-driven EvoMCP evolution pipeline used to generate and replay the current samples.

Guidelines:

- The answer [\[NA\]](#) means that the paper does not include experiments.
- If the paper includes experiments, a [\[No\]](#) answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.

- (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The released repository includes the generator, evaluator, configs, trace-bundle schema, report scripts, and a collaborator runbook with slot checks, EvoMCP commands, DAVIS background preparation, seedbank selection, and report regeneration commands.

Guidelines:

- The answer [NA] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The paper specifies the operation vocabulary, difficulty axes, seed-based generation, DAVIS video-temporal domain, EvoX/GEPA mutation surfaces, seedbank policy, and model evaluation modes. No model training is performed.

Guidelines:

- The answer [NA] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Reported video-temporal metrics include means and population standard deviations over response rows; the draft explicitly marks the full-pipeline leaderboard as incomplete where evidence is not yet sufficient.

Guidelines:

- The answer [NA] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The released evolution runbook records local versus remote EvoMCP execution, CLI-backed model lanes, budget ownership, artifact locations, and the non-expensive validation commands; no GPU training run is required for the reported results.

Guidelines:

- The answer [NA] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: This is foundational ML reasoning research without human subjects or sensitive data and conforms to the NeurIPS Code of Ethics.

Guidelines:

- The answer [NA] means that the authors have not reviewed the NeurIPS Code of Ethics.

- If the authors answer [No] , they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The work is a diagnostic benchmark for multimodal agents. The paper emphasizes reproducible failure analysis, fair solvability gates, and trace-based auditing rather than deployment of a new model.

Guidelines:

- The answer [NA] means that there is no societal impact of the work performed.
- If the authors answer [NA] or [No] , they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [NA]

Justification: The reasoning model poses no special misuse risk beyond standard research releases.

Guidelines:

- The answer [NA] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Dataset licenses are cited: MATH (MIT), HumanEval (MIT), MBPP (CC-BY-4.0), and GORE (authors’).

Guidelines:

- The answer [NA] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: We release Spectral-R1 training code, the spectral verifier implementation, and fine-tuned checkpoints with documentation.

Guidelines:

- The answer [NA] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [NA]

Justification: No human subjects or crowdsourcing are involved; all evaluations use automated benchmarks.

Guidelines:

- The answer [NA] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.

- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [NA]

Justification: No human subjects are involved, so IRB approval is not applicable.

Guidelines:

- The answer [NA] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLMs (Qwen-2.5) are central to the method and are described throughout Sections 4–5.

Guidelines:

- The answer [NA] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.